

Comprehensive Lesson: SQL Window Functions

Based on the screenshots provided, this lesson combines the theory of SQL Window Functions with practical syntax, examples, use cases, interview tips, and hands-on challenges.

1. What are SQL Window Functions?

A **Window Function** performs calculations across a set of rows that are related to the current row.

Unlike traditional aggregate functions, window functions:

- ✓ Do not collapse rows into a single result
- ✓ Return a value for every row
- ✓ Can compare rows within a logical group
- ✓ Enable advanced analytics directly in SQL

Traditional Aggregate

```
SELECT Department,  
AVG(Salary) AS AvgSalary  
FROM Employees  
GROUP BY Department;
```

Result:

Department	AvgSalary
IT	50000
HR	45000

Rows are collapsed.

Window Function

```
SELECT EmployeeName,  
Department,  
Salary,  
AVG(Salary) OVER(PARTITION BY Department) AS AvgDepartmentSalary  
FROM Employees;
```

Result:

Employee	Department	Salary	AvgDepartmentSalary
John	IT	55000	50000
Sarah	IT	45000	50000
Tom	HR	40000	45000
Amy	HR	50000	45000

Rows remain visible.

2. Categories of Window Functions

The first screenshot divides window functions into three major groups.

A. Aggregate Window Functions

These perform calculations over a window of rows.

Functions:

COUNT()

SUM()

AVG()

MIN()

MAX()

``

Example

```
SELECT  
EmployeeID,  
Salary,  
AVG(Salary) OVER() AS CompanyAverage  
FROM Employees;
```

B. Ranking Functions

Used for ranking rows within a dataset.

Functions:

```
ROW_NUMBER()  
RANK()  
DENSE_RANK()  
PERCENT_RANK()  
CUME_DIST()  
NTILE()  
,
```

Example Dataset

Employee	Salary
A	10000
B	9000
C	9000
D	8000

ROW_NUMBER()

Assigns unique sequence numbers.

```
SELECT Employee,  
Salary,  
ROW_NUMBER()
```

```
OVER(ORDER BY Salary DESC) AS RowNum  
FROM Employees;
```

Result

Employee	Salary	RowNum
A	10000	1
B	9000	2
C	9000	3
D	8000	4

RANK()

Allows ties and skips numbers.

```
SELECT Employee,  
Salary,  
RANK()  
OVER(ORDER BY Salary DESC) AS RankValue  
FROM Employees;
```

Result

Employee	Salary	Rank
A	10000	1
B	9000	2
C	9000	2
D	8000	4

Notice rank 3 is skipped.

DENSE_RANK()

No rank gaps.

```
SELECT Employee,  
Salary,  
DENSE_RANK()  
OVER(ORDER BY Salary DESC) AS DenseRank  
FROM Employees;
```

Result

Employee	Salary	DenseRank
A	10000	1
B	9000	2
C	9000	2
D	8000	3

NTILE()

Splits rows into groups.

```
SELECT Employee,  
Salary,  
NTILE(4) OVER(ORDER BY Salary DESC) AS Quartile  
FROM Employees;
```

Used for quartile analysis.

C. Value (Analytical) Functions

Functions that look at neighbouring rows.

From the screenshot:

```
LEAD()  
LAG()  
FIRST_VALUE()  
LAST_VALUE()
```

LAG()

Access previous row.

```
SELECT
OrderDate,
Sales,
LAG(Sales)
OVER(ORDER BY OrderDate) AS PreviousSales
FROM Orders;
```

Result

Date	Sales	PreviousSales
Jan	100	NULL
Feb	120	100
Mar	150	120

LEAD()

Look forward.

```
SELECT
OrderDate,
Sales,
LEAD(Sales)
OVER(ORDER BY OrderDate) AS NextSales
FROM Orders;
```

Result

Date	Sales	NextSales
Jan	100	120
Feb	120	150

Mar	150	NULL
-----	-----	------

FIRST_VALUE()

Returns first value in the window.

```
SELECT
Employee,
Salary,
FIRST_VALUE(Salary)
OVER(ORDER BY Salary DESC) AS HighestSalary
FROM Employees;
``
```

LAST_VALUE()

Returns last value in the frame.

```
SELECT
Employee,
Salary,
LAST_VALUE(Salary)
OVER(
ORDER BY Salary DESC
ROWS BETWEEN UNBOUNDED PRECEDING
AND UNBOUNDED FOLLOWING
) AS LowestSalary
FROM Employees;
``
```

3. Window Function Syntax

From the third screenshot:

```
AVG(Sales)
OVER(
PARTITION BY Category
```

```
ORDER BY OrderDate
ROWS UNBOUNDED PRECEDING
)
```

Breakdown:

```
Function() OVER()
```

The OVER clause defines the window.

General Structure

```
WindowFunction()
OVER
(
PARTITION BY column
ORDER BY column
FrameClause
)
```

4. PARTITION BY Clause

From the screenshot:

Divides the dataset into windows (partitions).

Think of it as a dynamic GROUP BY that doesn't collapse rows.

Example

```
SELECT
EmployeeName,
Department,
Salary,
AVG(Salary)
OVER(PARTITION BY Department) AS AvgSalary
FROM Employees;
```

Departments become separate windows.

IT Window

55000

45000

HR Window

40000

50000

5. ORDER BY Clause

According to the screenshot:

Sort the data in a window.

The ordering determines row sequence.

Example:

```
SELECT  
OrderDate,  
Sales,  
SUM(Sales)  
OVER(ORDER BY OrderDate)  
FROM SalesData;
```

Output is calculated in date sequence.

Without ORDER BY:

```
AVG(Sales) OVER()
```

Whole dataset used.

With ORDER BY:

```
AVG(Sales) OVER(ORDER BY OrderDate)
```

Running average.

6. Frame Clause

The second screenshot focuses heavily on the Frame Clause.

Frame clause determines:

Which rows inside the window will participate in the calculation.

Syntax

ROWS BETWEEN

lower_boundary

AND

upper_boundary

Frame Types

The screenshot shows:

ROWS

RANGE

ROWS

Counts actual physical rows.

ROWS BETWEEN 2 PRECEDING

AND CURRENT ROW

Means:

Current row

+

Previous 2 rows

RANGE

Works with values instead of row positions.

RANGE BETWEEN
UNBOUNDED PRECEDING
AND CURRENT ROW

Includes rows with equivalent ordered value.

7. Frame Boundaries

From screenshot:

Lower Boundary

CURRENT ROW
N PRECEDING
UNBOUNDED PRECEDING

Upper Boundary

CURRENT ROW
N FOLLOWING
UNBOUNDED FOLLOWING

Common Frames

Running Total

SUM(Sales)
OVER(
ORDER BY OrderDate
ROWS BETWEEN
UNBOUNDED PRECEDING
AND CURRENT ROW
)

Moving Average

Last 3 rows:

```
AVG(Sales)
OVER(
ORDER BY OrderDate
ROWS BETWEEN
2 PRECEDING
AND CURRENT ROW
)
```

Future Forecast Window

```
AVG(Sales)
OVER(
ORDER BY OrderDate
ROWS BETWEEN
CURRENT ROW
AND 2 FOLLOWING
)
```

Entire Partition

```
AVG(Sales)
OVER(
PARTITION BY Category
ROWS BETWEEN
UNBOUNDED PRECEDING
AND UNBOUNDED FOLLOWING
)
```

8. Real Business Scenarios

Running Revenue

```
SELECT
OrderDate,
Revenue,
SUM(Revenue)
OVER(
```

```
ORDER BY OrderDate
ROWS UNBOUNDED PRECEDING
) AS RunningRevenue
FROM Sales;
```

Customer Purchase Ranking

```
SELECT
CustomerID,
TotalSpent,
RANK()
OVER(ORDER BY TotalSpent DESC) AS CustomerRank
FROM Customers;
```

Month-over-Month Growth

```
SELECT
Month,
Sales,
Sales -
LAG(Sales)
OVER(ORDER BY Month) AS Growth
FROM MonthlySales;
```

Top Earners per Department

```
SELECT *
FROM
(
SELECT *,
ROW_NUMBER()
OVER(
PARTITION BY Department
ORDER BY Salary DESC
) AS RN
FROM Employees
) x
WHERE RN <= 3;
```

Common Interview Questions

Q1: Difference between GROUP BY and Window Functions?

GROUP BY	Window Function
Collapses rows	Keeps rows
One result per group	One result per row
Used for summaries	Used for analytics

Q2: ROW_NUMBER vs RANK vs DENSE_RANK

ROW_NUMBER : 1,2,3,4

RANK : 1,2,2,4

DENSE_RANK : 1,2,2,3

Q3: What does OVER() do?

Defines the window over which the function performs calculations.

Q4: When is a Frame Clause Required?

Typically with:

SUM()

AVG()

COUNT()

MIN()

MAX()

when performing running calculations or moving windows.

Practice Challenges

Challenge 1: Running Total

Given:

Date	Sales
Jan	100
Feb	150
Mar	200

Create a running total.

Expected:

100

250

450

Challenge 2: Top 2 Employees per Department

Use:

ROW_NUMBER()

PARTITION BY Department

Return only top 2 salaries per department.

Challenge 3: Month-over-Month Growth

Use:

LAG()

Calculate:

CurrentMonthSales - PreviousMonthSales

``

Challenge 4: Moving Average

Calculate a rolling 3-month average.

Hint:

ROWS BETWEEN 2 PRECEDING
AND CURRENT ROW

Challenge 5: Salary Quartiles

Split employees into four groups using:

NTILE(4)

Key Takeaways

1. **Window Functions** perform calculations without collapsing rows.
2. **OVER()** defines the window.
3. **PARTITION BY** creates logical groups.
4. **ORDER BY** controls row sequence.
5. **Frame Clauses** define the subset of rows used in calculations.
6. **Aggregate Window Functions:** SUM, AVG, COUNT, MIN, MAX
7. **Ranking Functions:** ROW_NUMBER, RANK, DENSE_RANK, NTILE
8. **Analytical Functions:** LAG, LEAD, FIRST_VALUE, LAST_VALUE
9. Window functions are essential for reporting, trend analysis, rankings, running totals, and business intelligence workloads.
10. A useful mental model is:

WINDOW FUNCTION

=

Function

+ OVER()

+ PARTITION BY

+ ORDER BY

+ Frame Clause

Mastering these four components allows you to solve most advanced SQL analytics problems encountered in data analysis, BI, data engineering, and SQL interviews.